

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME:	Cloud Computing and Big Data Analytics	COURSE CODE:	CSA15
---------------------	---	---------------------	-------

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Dave's Taxonomy: Precision (Level 3)
Question Count: 3

Total Marks: 30

Code of Conduct

I D Yashwanth(192472336 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D Yashwanth

Assessment Tasks

Case Study

1. Case Study 1: Smart Retail Inventory Management System

A retail chain implements a cloud-based inventory system connected to store devices. Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations.

2. Case Study 2: Cloud-Based Document Collaboration System

An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices. Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.

3. Case Study 3: Online Food Ordering Platform

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients. Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform. Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

Case Study Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application
Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis
Solution Design	20%	Solution is feasible, practical, and well justified	Reasonable solution with minor gaps	Basic solution	Unclear solution
Clarity	10%	Well-structured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized

Case Study 1: Smart Retail Inventory Management System

1. Understanding of the Case

The retail chain needs a centralized cloud-based inventory management system that connects multiple stores, POS systems, and management dashboards.

Technical Requirements

- Connect POS systems and dashboards to cloud services.
- Store product information, transaction records, and analytics data.
- Provide APIs for communication between clients and backend services.
- Support demand forecasting and inventory analytics.
- Provide secure access using VPNs, firewalls, and API gateways.
- Support multiple stores and users simultaneously.
- Provide scalability, availability, and data backup.

Business Requirements

- Maintain accurate inventory information.
- Reduce stock shortages and overstocking.
- Monitor sales from different stores in real time.
- Improve demand forecasting.
- Reduce manual inventory management.
- Support better business decisions using analytics.

2. Application of Cloud Concepts

Component	Purpose
-----------	---------

IaaS	Virtual machines and networking for application workloads
------	---

PaaS	Analytics and demand forecasting services
------	---

SaaS	Web-based inventory dashboards
------	--------------------------------

Cloud Storage	Product data, transaction logs, images and analytics datasets
---------------	---

API Gateway	Controls and manages API requests
-------------	-----------------------------------

VPN	Secure connection between stores and cloud
Firewall	Protects cloud resources from unauthorized traffic
Database	Stores product, sales and inventory information
Load Balancer	Distributes requests across application servers

3. Analysis

A centralized cloud system is better than maintaining separate inventory systems at every store because all stores can access updated information.

For example, when a product is sold at Store A, the transaction can be sent through an API to the cloud. The inventory database is updated, and the dashboard can immediately show the new stock level.

Important Trade-offs

- **Cloud storage vs local storage:** Cloud storage provides scalability and centralized access, while local storage may provide faster access but is harder to manage across many stores.
- **Security vs accessibility:** Strong authentication, VPNs, and API gateways improve security but can introduce additional configuration and management requirements.
- **Scalability vs cost:** Cloud resources can automatically scale during high-demand periods, but increased usage can increase cloud costs.

4. Proposed Solution Design

The architecture can follow:

POS Systems / Dashboards → API Gateway → Cloud Application Services → Database & Cloud Storage → Analytics & Forecasting

The API gateway receives requests from POS systems and dashboards. Backend services process the requests and communicate with the database and storage system.

Analytics services analyze historical sales data and generate demand forecasts. Security is provided using VPNs, firewalls, authentication, and access control.

5. Expected Benefits

- Real-time inventory monitoring.
- Improved demand forecasting.
- Reduced stock-outs.
- Centralized data management.
- Easy expansion to new stores.

- Better business analytics.
- Improved security and reliability.

Conclusion

The proposed cloud-based inventory system provides a scalable, secure, and centralized solution for retail inventory management. By combining IaaS, PaaS, SaaS, cloud storage, APIs, and security services, the company can improve operational efficiency and make better inventory decisions.

Case Study 2: Cloud-Based Document Collaboration System

1. Understanding of the Case

The enterprise requires a cloud-based collaboration platform similar to Google Docs where users can create, edit, share, and synchronize documents from different devices.

Technical Requirements

- Browser-based access.
- Real-time document editing.
- Multi-user collaboration.
- Distributed cloud storage.
- Version control.
- APIs for synchronization.
- High availability and low network latency.
- Encryption and identity management.
- Permission-based document sharing.

Business Requirements

- Allow employees to collaborate remotely.
- Reduce dependency on locally stored documents.
- Prevent loss of document versions.
- Improve productivity.
- Enable secure document sharing.
- Allow access from multiple devices.

2. Application of Cloud Concepts

Cloud Concept	Application
SaaS	Online document editing and collaboration application
Cloud Storage	Stores documents and previous versions
Web APIs	Synchronize changes between clients
Load Balancer	Distributes user requests
Distributed Infrastructure	Provides global availability
Identity Management	Authenticates users
Encryption	Protects documents during storage and transmission
Access Control	Determines who can view/edit documents

3. Analysis

Real-time collaboration requires changes made by one user to be quickly synchronized with other users.

For example, if User A modifies a document, the change is sent to the cloud through an API. The cloud service processes the update and synchronizes the change with other connected clients.

Important Trade-offs

- **Real-time synchronization vs complexity:** Real-time editing provides better collaboration but requires more complex synchronization mechanisms.
- **Distributed storage vs management:** Distributed storage improves availability but requires synchronization and consistency management.
- **Strong security vs convenience:** Strict access control improves security but may make sharing slightly less convenient.

4. Proposed Solution Design

The architecture can be:

Web Browser / Mobile Client → Web API → Collaboration Service → Distributed Cloud Storage

The identity management system authenticates users. After authentication, access permissions determine whether a user can view, edit, or share a document.

Documents are stored in distributed cloud storage, while version control maintains previous versions.

Encryption protects documents while they are being transferred and stored.

5. Expected Benefits

- Real-time collaboration.
- Access from anywhere.
- Automatic document backup.
- Version history.
- Secure document sharing.
- High availability.
- Reduced need for local storage.
- Improved employee productivity.

Conclusion

A cloud-based document collaboration platform is an effective SaaS solution because users can access the application through browsers without managing the underlying infrastructure. Distributed storage, APIs, encryption, identity management, and access control provide a secure and highly available collaboration environment.

Case Study 3: Online Food Ordering Platform

1. Understanding of the Case

The startup requires a scalable online food ordering platform that supports web and mobile users. Customers should be able to browse menus, place orders, and receive order-related information.

Technical Requirements

- Web and mobile client support.
- RESTful APIs.
- Cloud-based backend services.
- Secure cloud storage.
- Virtual machines.
- Managed Kubernetes.
- Authentication.
- HTTPS communication.
- Role-based access control.

- Scalability and high availability.

Business Requirements

- Allow customers to order food online.
- Support restaurants and administrators.
- Handle large numbers of simultaneous users.
- Provide reliable order processing.
- Protect customer information.
- Scale during peak ordering periods.

2. Application of Cloud Concepts

Component	Purpose
Web/Mobile Clients	Customer and restaurant access
REST API	Communication between frontend and backend
Virtual Machines	Run backend workloads
Kubernetes	Container orchestration and scaling
Cloud Storage	Store menu images, invoices and customer data
Load Balancer	Distributes incoming traffic
Authentication Tokens	Verify users
HTTPS	Encrypts communication
RBAC	Provides role-based permissions
Database	Stores users, menus and orders

3. Analysis

The food ordering system can experience sudden increases in traffic during lunch, dinner, weekends, and special offers.

Kubernetes can automatically scale application containers when demand increases. This allows the system to handle more users without requiring permanent high-capacity infrastructure.

For example:

Customer → Mobile/Web App → REST API → Backend Services → Database/Cloud Storage

The backend processes orders and stores relevant information in the cloud database.

Important Trade-offs

- **Kubernetes vs simple VMs:** Kubernetes provides better scalability and management for containerized applications but is more complex.
- **Scalability vs cost:** Automatic scaling handles peak traffic efficiently, but additional resources increase costs.
- **Security vs performance:** Authentication, HTTPS, and access control improve security but add processing and configuration overhead.

4. Proposed Solution Design

The proposed architecture is:

Customer Web/Mobile App

↓

Load Balancer

↓

RESTful API Gateway

↓

Kubernetes Cluster

↓

Backend Services

↓

Database + Cloud Storage

Authentication tokens verify users. HTTPS protects data during communication. RBAC ensures that customers, restaurant staff, delivery personnel, and administrators receive only the permissions appropriate to their roles.

Kubernetes manages application containers and allows services to scale according to demand.

5. Expected Benefits

- Supports large numbers of users.
- Automatic scalability.
- High availability.
- Secure customer information.
- Faster order processing.
- Easy deployment of new features.
- Efficient cloud resource utilization.

- Reliable operation during peak hours.

Conclusion

The cloud-based food ordering platform combines IaaS, containerization, Kubernetes, APIs, cloud storage, and security services to create a scalable and reliable application. Kubernetes is particularly useful because the number of users can change significantly throughout the day.